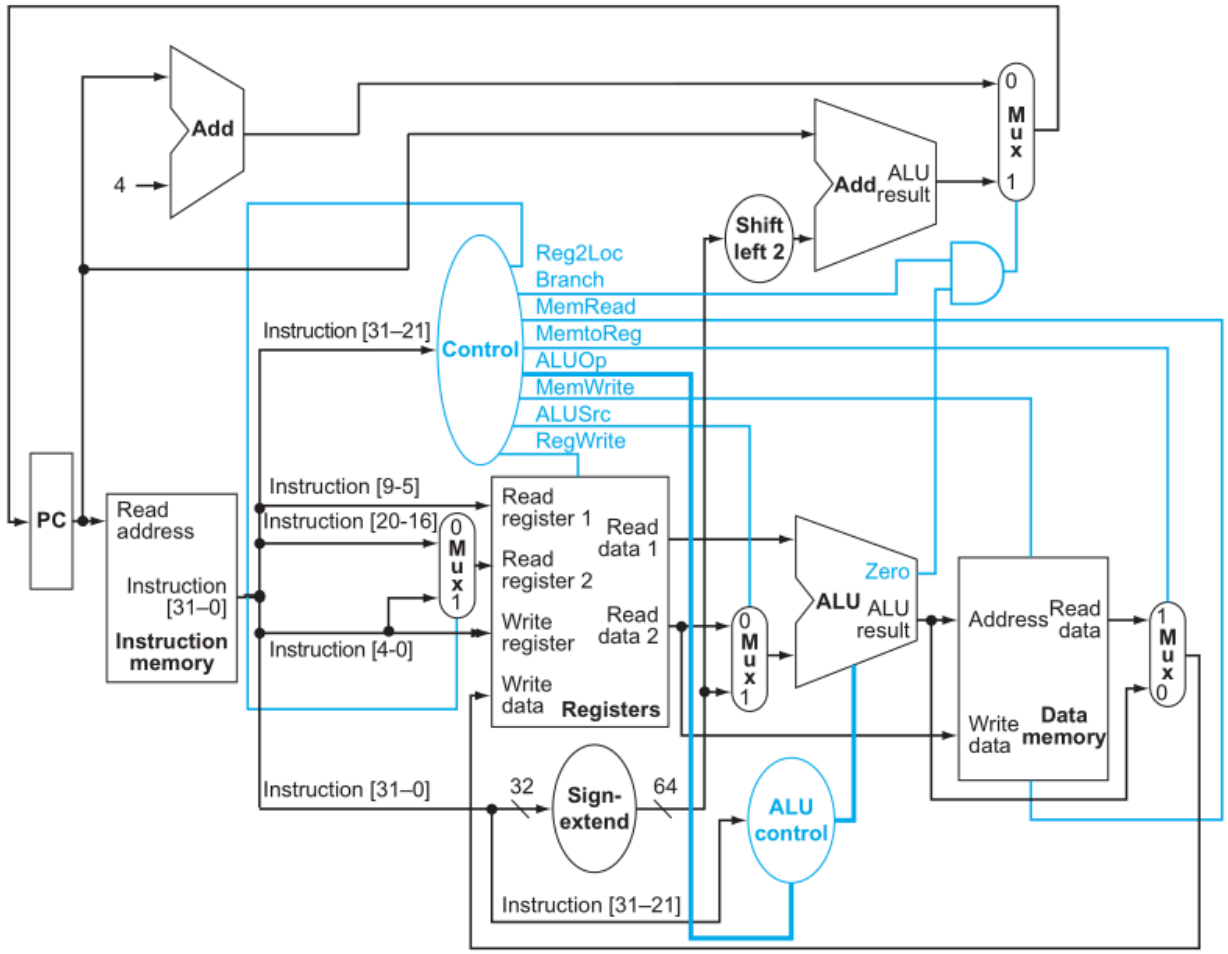
PRÁCTICO 9 - Implementación de la ISA

Gran parte de los ejercicios están basados en la Figura 4.17 "*The simple datapath with the control unit*" del libro COaD-LEGv8 que se copia abajo.



Instrucciones implementadas:

Instruction	ALUOp	Instruction operation	Opcode field	Desired ALU action	ALU control input
LDUR	00	load register	XXXXXXXXXXX	add	0010
STUR	00	store register	XXXXXXXXXXX	add	0010
CBZ	01	compare and branch on zero	XXXXXXXXXXX	pass input b	0111
R-type	10	ADD	10001011000	add	0010
R-type	10	SUB	11001011000	subtract	0110
R-type	10	AND	10001010000	AND	0000
R-type	10	ORR	10101010000	OR	0001

Ejercicio 1:
Marque con una barra invertida e indique el número de bits que representa cada una de las líneas del *data path*.

Ejercicio 2:
Considerando la siguiente distribución de instrucciones en un programa:

R-type	LDUR	STUR	CBZ
52%	25%	10%	13%

2.1) Qué porcentaje de todas las instrucciones utiliza `data memory`?
2.2) Qué porcentaje de todas las instrucciones utiliza `instruction memory`?
2.3) Qué porcentaje de todas las instrucciones utiliza el `sign extend`?

Ejercicio 3:
Complete la tabla con el estado de las señales **sin mirar el libro**.
Indique con `X` las condiciones no-importa.

Instr	Reg2Loc	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	AluOp1	AluOp0
R-type									
LDUR									
STUR									
CBZ									

Ejercicio 4:
Cuando se fabrican los chips de silicio, defectos en los materiales y errores en la fabricación pueden generar circuitos defectuosos. Un defecto común es que un cable de señal se rompa y siempre registre un '0' lógico. Esto se conoce comúnmente como "*stuck-at-0 fault*".

4.1) ¿Qué instrucciones operarían de forma incorrecta si el cable `MemToReg` está atascado en '0'?
4.2) ¿Qué instrucciones operarían de forma incorrecta si el cable `ALUSrc` está atascado en '0'?
4.3) ¿Qué instrucciones operarían de forma incorrecta si el cable `Reg2Loc` está atascado en '0'?

Ejercicio 5:
Agregando una compuerta en diagrama del *data path & control*, cambie la implementación de la instrucción `CBZ` a `CBNZ`.

Ejercicio 6:
En este ejercicio analizaremos en detalle cómo se ejecuta una instrucción en el *single-cycle datapath*, asumiendo que la palabra de instrucción que ejecuta el procesador es: `0xf8014062`, dado que `PC=0x100`.

6.1) ¿Cuáles son las salidas de los bloques `Sign-extend` y `Shift left 2` para esta palabra de instrucción?
6.2) ¿Cuáles son los valores de entrada a la unidad `ALU control` para esta palabra de instrucción?
6.3) ¿Cuál es la nueva dirección en el `PC` después de ejecutar esta instrucción?
6.4) Mostrar los valores de las entradas y salidas de cada `Mux` durante la ejecución de esta instrucción. Para los valores que son salidas de `Registers`, utilizar "`Reg [Xn]`".
6.5) ¿Cuáles son los valores de entrada de la `ALU` y las dos unidades `Add`?
6.6) ¿Cuáles son los valores de todas las entradas del bloque `Registers`?

Ejercicio 7:
Agregar a la implementación de la ISA la instrucción de **salto incondicional** `B`, a partir de una nueva señal que sale de *Control*, denominada `UncondBranch`.

Ejercicio 8:
Suponiendo que los diferentes bloques dentro del procesador tienen las siguientes **latencias**:

I-Mem / D-Mem	Register File	Mux	ALU	Adder	Single gate	Register Read	Register Setup	Sign extend	Control	SI2
250 ps	150 ps	25 ps	200 ps	150 ps	5 ps	30 ps	20 ps	50 ps	50 ps	25 ps

8.1) ¿Cuál es la latencia si lo único que tuviera que hacer el procesador es `fetch` de instrucciones consecutivas?
8.2) Hay un modo alternativo de generar la señal `Reg2Loc` fácilmente de la instrucción sin tener que esperar la latencia de `Control`. Explique cómo sería.
8.3) ¿Cuál es la latencia si solo hubiera instrucciones de tipo `R`?
8.4) ¿Cuál es la latencia para `LDUR`?
8.5) ¿Cuál es la latencia para `STUR`?
8.6) ¿Cuál es la latencia para `CBZ`?
8.7) ¿Cuál es el mínimo periodo de reloj para esta implementación de la ISA? Indicar también la frecuencia de reloj correspondiente ($f = 1/t$).

Ejercicio 9:
Para la tabla de latencias del Ejercicio 8, indicar el porcentaje de aumento de la velocidad de procesamiento si quitamos de las instrucciones de carga y almacenamiento la posibilidad de desplazamiento por el operando inmediato, es decir todas las instrucciones de carga son de la forma `LDUR X0, [X1]`.

Ejercicio 10:
Supongamos que podemos construir una CPU con un ciclo de reloj variable, que se puede adaptar a la latencia de cada instrucción. Cuál sería la aceleración de esta nueva CPU sobre la anterior (Ejercicio 8) si el mix de instrucciones de un programa es el siguiente*:

R-type	LDUR	STUR	CBZ
52%	25%	10%	13%

Ejercicio 11:
Al implementar los circuitos `Control`, `ALU control` se utilizaron muchas **condiciones no-importa** para simplificar la lógica. Esto produce **efectos laterales**. Cuando `Instruction[31:21]` está en el rango `0x5B8-0x5BF`, obtenemos del `Control:`

Reg2Loc	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	AluOp1	AluOp0
1	0	0	0	0	0	1	1	1

Mientras que de `ALU control` tiene una implementación utilizando condiciones no importa que produce:

AluOp1	AluOp0	I[31:21]	Operation
1	1	0x5B8	0110
⋮	⋮	⋮	⋮
1	1	0x5BF	0110

Este es un típico caso de **instrucción no documentada** con un comportamiento no del todo claro. Si, además, en el módulo Sign-extend esta instrucción se interpreta como un formato CB, indicar que hace esta instrucción, asignarle un mnemónico y describir la operación, a fin de completar la fila correspondiente a la nueva instrucción en la *green card*.

Ejercicio 12 (opcional):
Mostrar cómo se implementan los módulos:

- **12.1)** `Shift-left-2`: 64 bits de entrada, 64 bits de salida.
- **12.2)** `Sign-extend`: 32 bits de entrada, 64 bits de salida.
- **12.3)** Obtenga una implementación de la **composición** (`Sign-extend; Shift-left-2`) en un único circuito.

Ejercicio 13 (opcional):
Muestre el circuito interno de la ALU que, a partir de los 64 bits de salida, produce la salida `Zero` (`Z`).